ПРАКТИЧЕСКАЯ РАБОТА № 3

Форматы данных

Выполнил: Макаров Семён

Группа: 23 ИС 41

Цель работы: на практике изучить текстовые и бинарные форматы сериализации данных (JSON, XML, YAML, MessagePack, CBOR, Protobuf), сравнить их размеры и читаемость, освоить передачу данных по сети TCP и понять принципы схемной эволюции.

1. Ответы на теоретические вопросы

Чем отличаются текстовые и бинарные форматы?
Текстовые форматы (JSON, XML, YAML) представлены в виде символов, поэтому их удобно читать и проверять вручную. Бинарные форматы (MessagePack, CBOR, Protobuf) кодируют значения в байты, обычно занимают меньше места и требуют специальных инструментов для просмотра.

Что такое схемная эволюция?
Схемная эволюция — возможность изменять структуру данных, например добавлять или удалять поля, сохраняя совместимость между версиями программ. В этой работе проверяется добавление поля email в Protobuf и чтение ранее сохранённых данных старой схемой.

2. Результаты выполнения и сравнение форматов

Для сравнения использован профиль из методички: id = 42, имя — «Анна Смирнова», возраст — 22 года, статус студента, три курса и адрес в Москве.

Формат	Размер (байт)	Читаемость (1–5)	Комментарии?
JSON	211	5	Нет стандартной поддержки
XML	283	4	Да
YAML	182	5	Да, через #

Формат	Размер (байт)	Читаем человеком?	Нужна схема?
JSON	211	Да	Нет
XML	283	Да	Нет
YAML	182	Да	Нет
MessagePack	157	Нет	Нет
CBOR	Не измерен*	Нет	Нет
Protobuf	117	Нет	Да

* CBOR не измерялся: библиотека cbor2 отсутствовала в среде выполнения. Значение не подменялось расчётной оценкой.

3. Передача данных по TCP

Проверена клиент-серверная передача JSON и Protobuf через локальное TCP-соединение. Клиент отправлял байт-идентификатор формата, длину сообщения (4 байта, сетевой порядок) и полезную нагрузку. Сервер принимал данные, выводил HEX, текстовое представление и декодированные данные, затем сохранял файл в received/.

4. Скриншоты выполнения

Снимки консольного вывода получены при фактическом запуске соответствующих операций в локальной среде выполнения.

step_json.py — фактический вывод

```
=== JSON ===
{
  "id": 42,
  "name": "Анна Смирнова",
  "age": 22,
  "is_student": true,
  "courses": [
    "Математика",
    "Физика",
    "Информатика"
  ],
  "address": {
    "city": "Москва",
    "zip": "101000"
  }
}
Размер: 253 байт
Десериализация успешна: Анна Смирнова
```

Рисунок 1 — Выполнение step_json.py

step_protobuf.py — фактический вывод

```
=== Protobuf ===
HEX: 082a1219d090d0bdd0bdd0b020d0a1d0bcd0b8d180d0bdd0bed0b2d0b0181620012a14d09cd0b0d182d0b5d0bcd0b0d182d0
b8d0bad0b02a0cd0a4d0b8d0b7d0b8d0bad0b02a16d098d0bdd184d0bed180d0bcd0b0d182d0b8d0bad0b032160a0cd09cd0bed18
1d0bad0b2d0b01206313031303030
Размер: 117 байт
Десериализация успешна: Анна Смирнова
```

Рисунок 2 — Выполнение step_protobuf.py

server.py — приём JSON по TCP

```
=== Получено 211 байт, формат: json ===
HEX: 7b226964223a2034322c20226e616d65223a2022d090d0bdd0bdd0b020d0a1d0bcd0b8d180d0bdd0bed0b2d0b0222c202261
6765223a2032322c202269735f73747564656e74223a20747275652c2022636f7572736573223a205b22d09cd0b0d182d0b5d0bcd
0b0d182d0b8d0bad0b0222c2022d0a4d0b8d0b7d0b8d0bad0b0222c2022d098d0bdd184d0bed180d0bcd0b0d182d0b8d0bad0b022
5d2c20226164647265737223a207b2263697479223a2022d09cd0bed181d0bad0b2d0b0222c20227a6970223a202231303130303
0227d7d
TEXT: {"id": 42, "name": "Анна Смирнова", "age": 22, "is_student": true, "courses": ["Математика", "Физик
а", "Информатика"], "address": {"city": "Москва", "zip": "101000"}}
DATA: {'id': 42, 'name': 'Анна Смирнова', 'age': 22, 'is_student': True, 'courses': ['Математика', 'Физик
а', 'Информатика'], 'address': {'city': 'Москва', 'zip': '101000'}}
[СОХРАНЕНО] received/message_json.json (211 байт)
```

Рисунок 3 — Сервер принимает JSON: текст читается

server.py — приём Protobuf по TCP

```
=== Получено 117 байт, формат: protobuf ===
HEX: 082a1219d090d0bdd0bdd0b020d0a1d0bcd0b8d180d0bdd0bed0b2d0b0181620012a14d09cd0b0d182d0b5d0bcd0b0d182d0
b8d0bad0b02a0cd0a4d0b8d0b7d0b8d0bad0b02a16d098d0bdd184d0bed180d0bcd0b0d182d0b8d0bad0b032160a0cd09cd0bed18
1d0bad0b2d0b01206313031303030
TEXT: <не текст, бинарные данные>
DATA: {'id': 42, 'name': 'Анна Смирнова', 'courses': ['Математика', 'Физика', 'Информатика']}
[СОХРАНЕНО] received/message_protobuf.pb (117 байт)
```

Рисунок 4 — Сервер принимает Protobuf: бинарные данные



Рисунок 5 — Чтение старого сообщения новой схемой с полем email

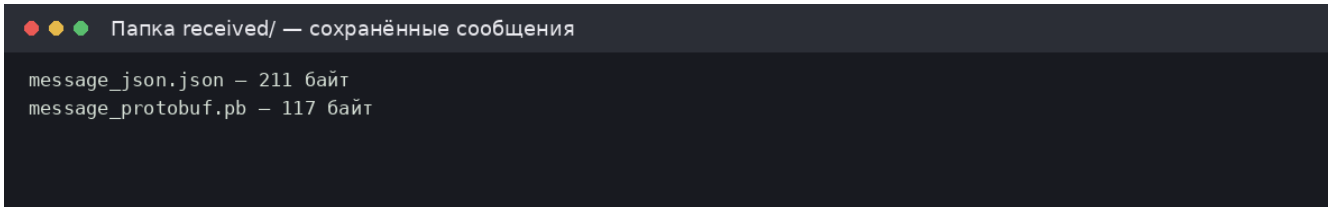


Рисунок 6 — Содержимое папки received/

5. Выводы

1. Почему Protobuf и MessagePack оказались меньше JSON?
В JSON имена полей и структура записываются текстом, поэтому повторяющиеся названия и служебные символы увеличивают размер. MessagePack и Protobuf используют компактное бинарное представление; Protobuf дополнительно использует номера полей из схемы.

2. В каком формате данные было легче всего читать вручную?
Проще всего читать YAML: он использует отступы и короткую запись структур, поэтому данные выглядят компактно и близко к обычному тексту. JSON также хорошо читается, но содержит больше скобок, кавычек и разделителей.

3. Почему JSON не поддерживает схемную эволюцию, а Protobuf поддерживает?
JSON сам по себе не задаёт обязательную формальную схему и правила совместимости версий: обработка новых полей зависит от программы. Protobuf использует отдельную схему с типами и постоянными номерами полей; при добавлении нового поля старые данные можно прочитать, а отсутствующее поле получает значение по умолчанию.

6. Примечания к выполнению

Фактически выполнены сериализация и десериализация JSON и Protobuf, TCP-передача этих двух форматов, сохранение файлов и проверка схемной эволюции. XML, YAML и MessagePack использованы для измерения размера. CBOR в данной среде не запускался из-за отсутствия библиотеки cbor2, поэтому его размер оставлен без выдуманного значения.

Команды из методички для запуска проекта: python step_json.py; python step_protobuf.py; python server.py; python client.py. Для TCP-теста сервер запускается перед клиентом.